Joshua Dalton
IT 505

1. What is an object and how are objects typically used, both in concept and in practice? (This question could result in several books if answered in detail, so keep your response at a very high level, as if you'd been asked this question on a timed exam.)

An object is a way of grouping related data and functions together. Objects are used to represent things in a program and to make it easier to organize and work with their respective information.

2. What is meant by the term member in the context of an object? Distinguish between data members and code members. How do these concepts relate to the commonly used terms property and method? What are the implications of TypeScript's first-class functions on this distinction?

A member is something that "belongs" to an object.

Data members = Stores information and are usually called properties

Code members = Functions called methods.

Considering TS supports first-class functions, functions can also be stored as properties, so the difference between properties and methods doesn't seem to always be clear.

3. What is the syntax for an object literal and when might it be useful?

It uses curly braces {} with properties written as name and value pairs. It's useful when you want to create an object without defining a class first. For example:

let player = { name: "Maye", number: 10 };.

4. There are two common ways of accessing a specific property or method of an object. Describe them both and explain when you might use one versus the other.

You can access properties and methods using dot notation or bracket notation. Dot notation is simpler and is used when you know the property name, while bracket notation is useful when the property name is stored in a variable or contains spaces or special characters.

5. What is destructuring in the context of objects? What is the syntax for employing it? When might it be useful?

Destructuring is used to let you take properties from an object and separately store them in variables. You use curly braces with the property names. Example:

const {name, number} = player;

This is useful when you need to access multiple properties without you typing the object name every time.

6. What is an iterable object? Provide at least two examples. What control structure would you use if you needed to iterate over the members in such an object? What control structure would you use if you needed to iterate over the members in an object that was not iterable?

They are objects you can loop through one value at a time. Arrays and strings would be two examples. You can use a for...of loop for iterable objects, while a regular object can be looped through using for...in.

7. Distinguish between the terms class and instance in the context of objects. How are these terms related?

A class is a blueprint that defines what properties and methods an object will have. An instance is an actual object created from that class. You can create multiple instances from the same class, and each one can have its own values.

8. When defining a specific instance of an object, must you first define a class? Why or why not? If a class is not required, is it at least possible? If so, provide a simple example demonstrating the basic syntax for defining a class.

No. You don't have to define a class first because you can create objects using object literals. You can define a class and use new to create an instance of it, however. Example:

```
class Player {
    name: string;
    constructor(name: string) {
        this.name = name;
    }
}
let player1 = new Player("Maye");
```

9. What convention is commonly followed when naming classes?

PascalCase. The first letter of every word is capitalized.

10. How do you define the properties of an object in an object literal? In a class definition? If there are both properties and methods being defined, where does convention dictate the properties should be defined?

In an object literal, properties are defined using names and values separated by colons. For a class, properties are declared with their names and types, and can also have starting values. Properties are usually defined at the top of the class before the methods.

Joshua Dalton
IT 505

11. How do you define the methods of an object in an object literal? In a class definition?

For an object literal, methods can be defined as functions assigned to properties. In a class, methods are defined by writing the method name, parentheses, and curly braces containing the code.

12. Within the body of a method, what does the special variable named this reference?

It references the object that the method is being called on and it allows the method to access that object's properties and other methods.

13. What is an object type and how are object types defined?

An object type describes what properties and methods an object is expected to have and what types they use. Example:

type Player = { name: string; number: number };.

14. What is an interface and how are interfaces defined? How are interfaces related to object types and function types? What convention is commonly followed when naming interfaces?

An interface defines the "structure" that an object is expected to follow. It uses the interface keyword followed by a name and curly braces containing its members. Interfaces are similar to object types and can also describe function types. Interface names use PascalCase.

15. What is the relationship between a class and strict mode?

JavaScript automatically runs code inside class definitions in strict mode, even if strict mode isn't enabled for the rest of the program. This means certain "unsafe" actions aren't allowed and some mistakes will cause errors instead of being ignored.

16. Explain the concept of a class's constructor. What is the most common reason for a constructor accepting arguments?

A constructor is a method that runs automatically when a new instance of a class is created. It is usually used to set the object's starting properties and arguments allow you to give each instance different values.

17. TypeScript offers a constructor shorthand that can be used to reduce the number of lines of code in many class definitions. Explain this syntax.

TS lets you put access modifiers in front of constructor parameters. This automatically creates and initializes the properties, so you don't have to declare and assign them separately. Example:

constructor(public name: string, public number: number) {}.

Joshua Dalton
IT 505

18. What are getters and setters and why are they useful? Provide a simple example illustrating the use of both.

Getters and setters are methods used to read and change properties. A getter returns (gets) something and a setter (sets) changes something. They are useful because you can control how properties are accessed or updated.

```
class Player {
    private _number: number = 0;
    get number(): number {
        return this._number; //returns the players number
    }
    set number(value: number) { //takes an argument and changes the players number
        this._number = value;
    }
}
```

19. What is a static property or method? Why are they useful? Is their use supported by TypeScript? If so, provide a simple example.

A static property belongs to the class itself. They are useful for information or functions that should be shared by the whole class. TS supports them using the static keyword.

```
class Player {
    static team: string = "Patriots";
}
console.log(Player.team);
```

20. What are optional properties, why are they useful, and what syntax is used to create them?

They are properties that an object doesn't need to include. They are useful when some information might not be needed or if you might not have all the information. In TS, you make a property optional by putting a question mark after its name. Example:

age?: number.

21. What are readonly properties, why are they useful, and what syntax is used to create them?

They are properties that can't be reassigned after they are initialized. They are useful for values that shouldn't change. Example:

readonly id: number.

Joshua Dalton
IT 505

22. What is inheritance in the context of object-oriented programming and why is it useful? Explain the concept of a subclass and provide a brief explanation of how subclasses might be used. Provide a simple example.

Inheritance lets one class “reuse” properties and methods from another class. A subclass "extends” a parent class and adds its own properties and methods. This is useful because you don't have to rewrite code that multiple classes share.

```
class Player {
    constructor(public name: string) {}
}
class Quarterback extends Player {
    throwPass() {
        console.log("Pass thrown");
    }
}
```

23. What must you do in a subclass’s constructor, and what syntax do you use to do it?

You have to call super() before using it. This runs the parent class's constructor and allows the subclass to pass values to it. Example:

```
super(name);
```

24. What does it mean for a subclass to override a method inherited from its superclass? How is this accomplished syntactically?

Overriding means a subclass creates its own version of a method that it inherited from its parent class. You do this by defining a method with the same name in the subclass. TS also supports the override keyword to make sure the method exists in the parent class.

25. What is an abstract class? What is the syntax for defining an abstract class? What can an abstract class contain that a non-abstract class cannot, and why? What is the associated syntax?

An abstract class is a class that can't be used to create objects directly. It is defined using “abstract class” and is meant to be extended by other classes. Different to a regular class, it can have abstract methods or properties that don't have an implementation yet. This lets subclasses to provide their own implementations.

```
abstract class Player {
    abstract play(): void;
}
class Quarterback extends Player {
    play(): void {
```

```
        console.log("Throw");
    }
}
```

26. Is there a mechanism available for determining whether an object belongs to a specified class or one of that class's subclasses? If so, what is the syntax?

Yes. You use the instanceof operator. Example:

player1 instanceof Player.

27. Is it possible to indicate that specific properties and methods of a class should be private? Public? Protected? Explain and provide a simple example demonstrating what is allowable. Is there a difference in this regard between JS and TS? If so, explain the details.

Yes to all three. Public members can be accessed anywhere, private members can only be accessed inside their class, and protected members can be accessed inside the class and subclasses. TS checks these restrictions during type checking, while JS uses # for private fields that are enforced when the program runs.

```
class Player {
    public name: string = "Maye";
    private rating: number = 95;
    protected team: string = "Patriots";
    #secret = 1;
}
```

28. What is polymorphism in the context of object-oriented programming and why is it useful? Is its use supported by TypeScript? If so, provide a simple example.

It lets different objects use the same method but have different results. TS supports polymorphism, and it is useful because you can work with different types of objects without having to write separate code for each one.

```
class Player {
    play(): void {
        console.log("Playing");
    }
}
class Quarterback extends Player {
    override play(): void {
        console.log("Passing");
    }
}
let players: Player[] = [new Player(), new Quarterback()];
for (let p of players) {
```

```
    p.play();
}
```

29. Describe some ways that the clean code philosophy could be meaningfully applied to the material you've discovered in this unit.

Clean code can be applied by giving classes, properties, and methods clear names, keeping methods simple, and most importantly avoiding repeated code. Using interfaces and inheritance when needed can also make programs easier to understand and maintain. I would also try to keep each class focused on one purpose and only make properties public when necessary.